PROJETO REPLAY · DOCUMENTAÇÃO DE SISTEMA

Guia completo do que está implementado

Hardware do piloto + software em produção. Somente o que já existe no código e no equipamento ligado. Sem roadmap.

Versão do agente na borda: 0.4.5 · Worker Cloudflare · D1 `replay-db` · R2 `replay-clips`

Base pública: `https://projeto-replay.gabrieltenoriovaz-10a.workers.dev`

Repositório: `gabvaz/projeto-replay` · branch `main`

Documento gerado em 21/10/2026 (BRT).

O produto, em uma frase

Jogador aperta um botão na quadra. Os ~20 segundos anteriores (buffer circular na Raspberry Pi) mais 3 segundos depois viram um MP4, sobem para a nuvem e aparecem na galeria do QR. Os vídeos expiram em 7 dias.

Isto não é NVR, não é live, não é transcodificação na nuvem. O gatilho é local (ESP32 + Pi). A nuvem só recebe, guarda, lista e entrega o arquivo.

Sumário

1. Visão geral e papéis
2. Hardware do piloto (topologia, pinos, IPs)
3. Firmware do botão (ESP32)
4. Agente na Pi (buffer, clip, upload, serial, systemd)
5. Instalação e operação da borda
6. Nuvem: Worker, D1, R2, crons
7. APIs públicas, edge, admin e portal
8. Front: rotas e comportamento
9. Modelo de dados
10. Autenticação e segurança
11. Observabilidade e alertas
12. Mapa de arquivos no repositório
13. Como usar o piloto no dia a dia

1. Visão geral e papéis

Três camadas, cada uma com um trabalho só:

Camada	Onde	Faz	Não faz
Quadra	Câmera PoE + ESP32 + botão	Imagem contínua RTSP; aperto físico	Upload, galeria, internet
Borda	Raspberry Pi 3 B+	Buffer, concat, MP4, fila, PUT no Worker, heartbeat	Servir o site; transcodificar na nuvem
Nuvem	Cloudflare Worker + D1 + R2	Slot de upload, storage, galeria, painel, portal, retenção	Falar com a câmera ou com o ESP

O Worker não alcança a câmera. A API local do agente escuta só `127.0.0.1`. Quem dispara o clip é: botão USB-serial, `POST /local/replay` na Pi, tecla no terminal, ou `T` no serial.

Fuso de galeria e retenção: Brasilia UTC-3 fixo (sem horario de verao). Datas na UI sao "hoje" em BRT, nao no relogio do celular.

Retenção padrão: `RETENTION_DAYS=7`. Clip ganha `expires_at = recorded_at + 7 dias` no pedido de slot. Cron diário apaga D1 + objetos R2 vencidos.

2. Hardware do piloto

Uma quadra em operação contínua: `demo01`. Hostname da Pi: `replay-quadra1`. Modelo: Raspberry Pi 3 Model B Plus Rev 1.4, ~905 MiB RAM, aarch64, Raspberry Pi OS.

2.1 Topologia

```
Internet -- Wi-Fi (wlan0) -- Raspberry Pi 3 B+                              IP LAN tipico: 192.168.
0.13                              |                               | cabo Ethernet (eth0)
                  v                           switch PoE                                | cabo PoE
                        v                     camera IP 192.168.1.108:554ESP32 -- USB -- Pi   (/dev
/replay-button -> ttyACM0 neste piloto)Botao NA entre GPIO 4 e GNDLED    GPIO 2 (LED da DevKit) + G
ND, resistor 220-330 ohm se for LED externo
```

Duas redes na Pi de propósito:

- wlan0 - internet: SSH, heartbeat, upload do MP4 para o Worker.
- eth0 - só câmera, via switch. A câmera não tem rota para a internet.

O RTSP nunca atravessa Wi-Fi neste piloto. Isso é o que deixa o buffer estável.

2.2 Câmera

Item	Valor no piloto
Endereço	`192.168.1.108`
Protocolo	RTSP sobre TCP (porta 554)
Path main	`/cam/realmonitor?channel=1&subtype=0`
Path extra (não usado no agent atual)	`/cam/realmonitor?channel=1&subtype=1`
Auth	user/senha na URL; `@` na senha vai percent-encoded (`%40`)
Alimentação	PoE 802.3af pelo switch
Áudio no replay	Descartado (`-an` / `-map 0:v:0`)

A URL RTSP no `edge.env` precisa de aspas se tiver `&`, senão o systemd corta o valor.

```
RTSP_URL='rtsp://admin:SENHA_ENCODED@192.168.1.108:554/cam/realmonitor?channel=1&subtype=0'RTSP_TRA
NSPORT=tcp
```

2.3 Raspberry Pi - o que está instalado

Item	Detalhe
Usuário de SSH	`pi`
Usuário do serviço	`replay` (system, grupos `dialout` e `video`)
Node	22.x em `/usr/local/bin/node`
ffmpeg	pacote Debian; copy H.264 do RTSP, concat, thumb
Serviço	`replay-agent.service` - `Restart=always`, `RestartSec=3`, `MemoryMax=500M`, `Nice=-5`
Watchdog	`dtparam=watchdog=on` no config.txt + `RuntimeWatchdogSec=15`
journald	`SystemMaxUse=50M` (protege o cartão SD)
Buffer	`/dev/shm/replay` (RAM, não SD)
Outbox	`/var/lib/replay/outbox` (persiste reboot)

Há suporte no código a uma segunda instância do agent (outra `EDGE_PORT`, outro `COURT_KEY`, outro `BUFFER_DIR`). Isso foi usado em teste. O systemd instalado é um serviço / uma quadra. Reboot não sobe segunda câmera.

2.4 ESP32 e botão

Função	Pino	Ligação
Botão	GPIO 4	NA entre GPIO 4 e GND. Pull-up interno. Nunca 3V3 no pino.
LED de estado	GPIO 2	LED da placa na DevKit típica, ou LED + 220-330 ohm para GND
USB	-	Serial 115200. Neste piloto: `ttyACM0` (USB nativo Espressif VID `303a`)

Regra udev `edge/pi/99-replay-button.rules` cria `/dev/replay-button` para:

- CP210x - `10c4:ea60`
- CH340 - `1a86:7523`
- CH9102 - `1a86:55d4`
- ESP32-S3/C3 USB nativo - `303a:1001`

Grupo `dialout`, modo `0660`. O serviço do agent pertence a esse grupo.

3. Firmware do botão

Arquivo: `firmware/button/button.ino`. Arduino / ESP32. Serial 115200, uma mensagem por linha, `\n`.

3.1 Protocolo

Direção	Linha	Significado
ESP -> Pi	`BTN`	Aperto válido (debounce 50 ms, borda de descida)
ESP -> Pi	`HB`	Heartbeat a cada 10 s
Pi -> ESP	`ACK`	Resposta ao HB (Pi vivo)
Pi -> ESP	`REC`	Aceitou o aperto, está gravando
Pi -> ESP	`OK`	Clip gerado (upload pode ainda estar na fila)
Pi -> ESP	`ERR`	Falhou gerar
Pi -> ESP	`BUSY`	Já gravando ou cooldown
Debug -> ESP	`T`	Mesmo efeito do botão (serial monitor)

3.2 Constantes

Constante	Valor
`BUTTON_PIN`	4
`LED_PIN`	2
`DEBOUNCE_MS`	50
`HB_EVERY_MS`	10 000
`REPLY_TIMEOUT_MS`	2 000 (sem REC/BUSY após BTN -> NO_HOST)
`RECORDING_TIMEOUT_MS`	60 000
`FEEDBACK_MS`	1 500 (mostra ERR)
`HOST_TIMEOUT_MS`	3×HB + 2 s ~ 32 s sem ACK -> LED apaga

3.3 LED

Estado	LED
READY	Aceso fixo
WAITING_REPLY / RECORDING	Pisca ~4 Hz
SHOW_OK	Aceso 2 s + 3 piscadas lentas
SHOW_ERR	Pisca rápido 1,5 s
NO_HOST	Apagado

Aperto em `NO_HOST` ainda envia `BTN` (a Pi pode ter voltado). Só ignora se já está em WAITING/RECORDING/feedback.

4. Agente na Pi

Arquivo: `edge/agent.mjs`. Sem dependências npm. Sobe com `/etc/replay/edge.env` (systemd) ou `edge/.env` (PC).

Se `EDGE_TOKEN` estiver vazio o processo recusa subir (não envia clip anônimo).

4.1 Variáveis de ambiente

Nome	Padrão no código / install	Função
`API_BASE`	URL do Worker	Base HTTPS dos PUT/heartbeat
`COURT_KEY`	`demo01`	Deve existir em `courts.public_key` da mesma arena do device
`EDGE_TOKEN`	-	Bearer do device; só o SHA-256 fica no D1
`RTSP_URL`	vazio	Vazio = `testsrc` 1280×720 sintético (dev)
`RTSP_TRANSPORT`	`tcp`	`udp` só se a cam recusar TCP
`EDGE_PORT`	8788	HTTP local 127.0.0.1
`PRE_ROLL_SEC`	20 (install) / 15 (código)	Segundos do passado no clip
`POST_ROLL_SEC`	3	Espera depois do aperto antes de concatenar
`SEGMENT_SEC`	2	Alvo do muxer de segmentos
`COOLDOWN_SEC`	5	Ignora apertos seguintes
`OUTPUT_FORMAT`	`story` no install.sh	`story` = re-encode 540×960; vazio = copy
`CAMERA_ROTATION`	0	0, 90 ou 270. Em story vira transpose; em copy vira metadata/display_rotation
`BUTTON_SERIAL`	`/dev/replay-button`	Só Linux
`BUFFER_DIR`	`/dev/shm/replay` no Linux	Segmentos .ts
`OUTBOX_DIR`	`/var/lib/replay/outbox`	MP4 + JSON + events.jsonl
`OUTBOX_MAX_MB`	200	Descarta os mais antigos se estourar (`queue_drop`)
`HEARTBEAT_SEC`	60	POST `/edge/heartbeat`
`OPEN_UI`	-	`1` abre o browser local (PC)
`FFMPEG_PATH`	-	Override do binário ffmpeg

4.2 Buffer circular

ffmpeg contínuo, vídeo copy, sem áudio, segmentos MPEG-TS:

```
-rtsp_transport tcp -timeout 5000000 -i RTSP_URL-map 0:v:0 -c:v copy -an-f segment -segment_time 2
-segment_wrap N -reset_timestamps 1-segment_format mpegts /dev/shm/replay/seg_%03d.ts
```

`SEGMENT_WRAP = ceil((PRE+POST+10)/SEGMENT) + 5`. Com pre 20 / post 3 / seg 2 ~ 22 arquivos.

`-timeout 5000000` é 5 s em microsegundos: câmera some -> ffmpeg sai -> supervisor relança.

Backoff de restart: 1, 2, 5, 10, 30 s. Se o processo durou mais de 60 s, o backoff zera. Cada saída gera evento `ffmpeg_restart`.

Watchdog de stall: se o ffmpeg está vivo mas nenhum `.ts` novo por `max(3×SEGMENT, 6)` s, manda SIGKILL (câmera "congelada").

Senha RTSP é mascarada no stderr antes de ir ao journald.

4.3 Montagem do clip

1. Marca `busy`, evento `press`, espera `POST_ROLL_SEC`.
2. Lista segmentos por mtime. Soma duração real (intervalo entre mtimes) porque o GOP da câmera faz cada .ts durar mais que 2 s.
3. Exige pelo menos 3 segmentos e cobertura >= metade de (pre+post). Senão: `buffer ainda frio`.
4. Concat ffmpeg `-f concat -safe 0`.
5. Se `OUTPUT_FORMAT=story`: crop central 9:16 (ou transpose 90/270) + `scale=540:960` + libx264 `ultrafast` CRF 26, 3 threads, yuv420p, `+faststart`, sem áudio.
6. Se vazio: `-c copy` + faststart (landscape nativo).
7. Thumb: um JPEG (largura 480) extraído do MP4.
8. Escreve `{clip_id}.mp4` + `{clip_id}.json` (clip_id, court_key, recorded_at) na outbox. JSON via write+rename atômico.

Gera UUID do clip na Pi. O slot no Worker é idempotente nesse id - queda de internet no meio do PUT não duplica.

4.4 Upload

1. `POST /edge/clips/upload-slot` com `Authorization: Bearer EDGE_TOKEN`, body `{ court_key, clip_id, recorded_at }`.
2. `PUT /edge/clips/:id/content` - corpo = bytes do MP4.
3. `PUT /edge/clips/:id/thumbnail` - JPEG.
4. Apaga a entrada da outbox; evento `clip_ok`.

Falha: backoff 5, 15, 30, 60, 120, 300 s. Evento `clip_fail`. A outbox sobrevive a reboot.

4.5 HTTP local (127.0.0.1)

Método	Path	Função
GET	`/`	UI mínima (botão de replay no browser da Pi)
GET	`/health`	JSON: ok, busy, court, api, buffer_alive, queue_len, pending_events
POST	`/local/replay`	Dispara o mesmo pipeline do botão
POST	`/local/serial-t`	Escreve `T\n` na porta (debug do firmware)

Teclado no TTY: Enter, Espaço ou R também disparam.

4.6 Serial USB - detalhe que evita reset eterno

Um único `openSync(path, "r+")`. Dois streams na mesma path erguem DTR duas vezes e o ESP fica em boot loop. Depois do open, pulso RTS/DTR via Python embutido (`releaseEspReset`) e `stty -hupcl` para não derrubar DTR ao fechar.

Primeiro chunk após conectar costuma ser lixo de boot (`rst:0x1 POWERON_RESET`) - o agent loga só esse primeiro pedaço. Linhas seguintes `HB` / `BTN` não poluem o journal.

4.7 Heartbeat (payload)

JSON aproximado enviado a cada 60 s, mais eventos pendentes (máx. 50 por request, 500 em disco):

- `agent_version` = `0.4.5`
- `temp_c`, `throttled` (vcgencmd)
- `load1`, `mem_free_mb`, `mem_total_mb`, `uptime_s`
- `buffer_alive`, `segments`, `busy`, `queue_len`
- `button_alive` (HB recente)
- `ffmpeg_restarts_1h`
- `events[]`: `press | clip_ok | clip_fail | ffmpeg_restart | boot | queue_drop` com `at` ISO e `detail` <= 200 chars

5. Instalação e operação da borda

5.1 install.sh

`sudo bash edge/pi/install.sh [--hostname ...] [--tailscale-authkey ...]`

Idempotente: atualiza agent, udev, unit, journald, watchdog. Não sobrescreve `/etc/replay/edge.env` se já existir.

Se `COURT_KEY` ou `EDGE_TOKEN` estiverem vazios, habilita o serviço mas não inicia.

Instala Tailscale com `--ssh` (acesso remoto sem abrir porta 22 na internet).

5.2 Criar device

```
$env:ADMIN_TOKEN="..."npm run admin:device -- --venue=venue-demo --name="Pi Arena Demo"
```

Imprime `EDGE_TOKEN=...` uma vez. Cole no `edge.env` e `systemctl restart replay-agent`.

5.3 Atualizar o agent de outro PC

```
powershell -File edge/pi/update.ps1 -PiHost replay-quadra1powershell -File edge/pi/update.ps1 -PiHo
st replay-quadra1 -Full
```

`-Full` reenvia install.sh, unit e udev. Sem `-Full` só troca `agent.mjs` e restart.

5.4 Comandos no Pi

```
systemctl status replay-agentjournalctl -u replay-agent -fcurl -s 127.0.0.1:8788/healthcurl -sS -X
POST http://127.0.0.1:8788/local/replaysudo nano /etc/replay/edge.envsudo systemctl restart replay-
agentls -l /dev/replay-button
```

5.5 Unit systemd (campos)

```
[Service]User=replayGroup=replaySupplementaryGroups=dialout videoEnvironmentFile=/etc/replay/edge.e
nvWorkingDirectory=/opt/replayExecStart=/usr/local/bin/node /opt/replay/agent.mjsRestart=alwaysRest
artSec=3MemoryMax=500MNice=-5KillMode=control-groupTimeoutStopSec=10StartLimitIntervalSec=0
```

6. Nuvem

6.1 Worker

Campo	Valor
Nome	`projeto-replay`
Entrada	`src/index.ts` (Hono)
Assets	`web/dist`, SPA `not_found_handling`
Worker-first	`/api/* /edge/* /admin/* /clips/* /courts/* /c/* /v/* /portal/* /a/* /brand/* /__scheduled`
D1	`replay-db` id `ef8f09c2-b761-40be-a856-42efa8c22bd1`
R2	`replay-clips` binding `CLIPS`
Compat date	2026-09-24

Vars no `wrangler.jsonc`:

Var	Produção atual
`RETENTION_DAYS`	`7`

`DEVICE_OFFLINE_SEC`	`180` (painel: sem HB = offline)
`SUPPORT_WHATSAPP`	vazio (esconde no portal)
`APP_URL`	URL workers.dev
`ALERT_EMAIL_FROM` / `TO`	vazios -> e-mail não sai, só `console.log`

Secrets (não no jsonc): `ADMIN_TOKEN`, `SESSION_SECRET`, `IP_HASH_SALT`.

Binding de e-mail (`send_email`) está comentado no wrangler até o domínio entrar no Email Routing.

Deploy: `npm run deploy` (= `vite build` + `wrangler deploy`).

Migrações: `npm run db:migrate:local` / `db:migrate:remote`. Até `0008_venue_public_key.sql`.

6.2 Objetos no R2

Key	Conteúdo
`courts/{courtId}/{clipId}.mp4`	vídeo
`courts/{courtId}/{clipId}.jpg`	thumb
`brand/{venueId}/logo-{rand}`	logo PNG/JPEG/WebP (SVG recusado)

Bucket não é público. Entrega só via Worker (`/clips/:id` com range 206, cache curto; thumb cache 1 dia; logo com `?v=` immutable).

6.3 Crons

Cron	Job	Faz
`*/5 * * * *`	`runAlerts`	Abre/fecha alertas. Offline de alerta = 10 min (mais lento que os 180 s do painel). Dedupa por (device, kind). E-mail só na mudança, se FROM/TO existirem.
`0 9 * * *` (06:00 BRT)	`runDaily`	Purge clips expirados e pending > 1 dia; apaga heartbeats > 7 d; events/alerts resolvidos > 90 d; `ensureInvoices`; resumo 24h (apertos -> clips).

Códigos de alerta do payload: `hot` (>=75 °C), `throttled`, `buffer`, `queue` (>3), `button`, `camera` (>=3 restarts ffmpeg/h), mais `offline`.

7. APIs

CORS em `/api/*`, `/courts/*`, `/clips/*`.

7.1 Público / jogador

Método	Path	Auth	Notas
GET	`/api/health`	-	`{ ok, service, core, trigger: "local-only" }`
GET	`/api/courts/:key`	-	quadra + brand + venue_key + retention + today BRT
GET	`/api/courts/:key/clips?date=YYYY-MM-DD`		clips ready do dia, máx. 300, hora BRT
GET	`/api/venues/:key`	-	arena pública + lista de quadras
GET	`/api/venues/:key/clips?date=&court=`		todas as quadras ou filtro; máx. 500
GET	`/api/clips/:id`	-	detalhe + brand + venue_key; 404 se expirado
POST	`/api/clips/:id/events`	-	body `{ type: "view" | "share" }`
GET	`/clips/:id`	-	stream; `?download=1` registra download se sem Range
GET	`/clips/:id/thumb.jpg`	-	JPEG
GET	`/brand/:venueId/logo?v=`	-	logo
GET	`/v/:id`	-	SPA + og:title/image p/ WhatsApp
GET	`/c/:key`	-	301 -> `/q/:key`
GET	`/courts/:publicKey/clips`	-	legado; front usa `/api/...`
GET	`/a/:token`	token do link	seta cookie, redirect `/arena`

Janela do dia na query de clips: `created_at` UTC convertido com `+3 hours` para BRT, `date` obrigatório `YYYY-MM-DD`.

Eventos de view/download/share: IP vira SHA-256(`ip|hojeBRT|IP_HASH_SALT`) - não rastreia entre dias.

7.2 Edge (Pi)

Middleware: Bearer = token do device, hash comparado com `devices.token_hash`, `revoked_at IS NULL`.

Método	Path	Notas

POST	`/edge/clips/upload-slot`	Idempotente por clip_id UUID. Court tem que ser da mesma venue do device. `recorded_at` só se ISO nos últimos 24 h.
PUT	`/edge/clips/:clipId/content`	Só se status ainda `pending`; senão 409. Marca `ready`, grava bytes, evento `created` com a hora do lance.
PUT	`/edge/clips/:clipId/thumbnail`	Máx. 1 MB
POST	`/edge/heartbeat`	Atualiza `last_seen_at`, `last_payload`, `agent_version`. Histórico em `heartbeats` só a cada 15 min ou se houver anomalia. Eventos -> `device_events`.

7.3 Admin (`/admin`, UI em `/painel`)

Bearer `ADMIN_TOKEN` (scripts) ou cookie `rs_admin` (HMAC 8 h, Secure, HttpOnly, SameSite=Strict). Mutação via cookie exige JSON (anti form CSRF). `POST /admin/session` é público (login).

Método	Path
POST/GET/DELETE	`/admin/session`
GET	`/admin/overview`
GET/POST	`/admin/venues`
GET/PATCH	`/admin/venues/:id`
POST/DELETE	`/admin/venues/:id/access-link`
PUT/DELETE	`/admin/venues/:id/contract`
POST	`/admin/venues/:id/courts`
PATCH	`/admin/courts/:id`
POST/GET	`/admin/devices`
GET	`/admin/devices/:id/events`
POST	`/admin/devices/:id/revoke`
GET/POST	`/admin/equipment`
PATCH	`/admin/equipment/:id`
GET	`/admin/equipment/:id/events`
GET	`/admin/invoices`
POST	`/admin/invoices/:id/pay|void|reopen`
GET/DELETE	`/admin/clips` · `/admin/clips/:clipId`

UI admin: Visão geral, Arenas (detalhe com abas resumo/quadras/dispositivos/equipamentos/financeiro/clips), Equipamentos, Financeiro, Dispositivos. Sem

auto-refresh na visão geral (query refetch só no foco). Métrica no card do device: taxa aperto->clip 7 dias + eventos recentes.

Status de arena: `onboarding` | `active` | `paused` | `churned`.

`public_key` de quadra: 3-32 chars `a-z0-9-`, único.

7.4 Portal da arena (`/portal`, UI em `/arena`)

Cookie de sessão a partir de `/a/:token`. Token do link é hasheado; dá para revogar. `noindex`.

Método	Path	UI
GET	`/portal/me`	-
GET	`/portal/overview`	`/arena`
GET	`/portal/courts`	`/arena/quadras` (QR, URL `/q/...`)
GET	`/portal/devices`	`/arena/status` (online/offline, último clip - sem token)
GET/DELETE	`/portal/clips`	`/arena/clips` (dono pode apagar)
GET	`/portal/invoices`	`/arena/faturas`
PATCH/PUT/DELETE	`/portal/branding` (+ logo)	`/arena/personalizar`
DELETE	`/portal/session`	sair

Marca: `public_name` <= 60; cores permitidas `#d7ff3a #ffd23f #ff9f43 #ff7a7a #ff7ac0 #b99aff #4dd4ff #3ee6a0`. Logo PNG/JPEG/WebP, magic-bytes checados, sem SVG (XSS).

Equipamentos no admin (inventário, não no portal): kinds `camera` | `pi` | `switch` | `esp32` | `button` | `adapter` | `other`; status `stock` | `installed` | `maintenance` | `lost` | `retired`.

Faturas: contrato com `monthly_cents` + `due_day` 1-28; cron/ensure gera período `YYYY-MM`; status `open` | `paid` | `void`.

8. Front público

React 19, Vite 8, Tailwind 4, React Router 7, TanStack Query. Fontes Inter + Barlow Condensed.

Rota	Página	Comportamento
`/`	Home	Formulário "código da quadra" -> `/q/...`
`/q/:key`	CourtPage	Hero, DayPicker, featured de hoje, HourGroup, poll 15 s se hoje, toast de clip novo. Nome da arena linka para `/g/{venue_key}`
`/q/:key/qr`	QrPage	Cartaz (lazy)
`/g/:key`	VenuePage	Todos os clips da arena; chips de quadra (`?court=`); badge da quadra no card

`/v/:id`	ClipPage	Player 9:16, baixar, share, "mais da quadra"
`/painel/*`	AdminApp	lazy
`/arena/*`	ArenaApp	lazy; queries sem refetch no foco

Clips novos: `isFresh` = hoje e horário local há menos de 10 min -> selo "Novo". ViewTransition thumb->player. Share dispara evento `share`.

Empty state ensina o botão. Footer: "Atualiza sozinho a cada 15s".

9. Modelo de dados (D1)

Migrações 0001-0008. Tabelas efetivas:

Tabela	Papel
`venues`	Arena. Campos extra: city, contato, status, notes, public_name, brand_color, logo_key, public_key (slug `/g`)
`courts`	Quadra. `public_key` único (QR)
`clips`	id UUID, court_id, object_key, thumb_key, status pending/ready, bytes, device_id, created_at (= hora do lance), ready_at, expires_at
`clip_events`	created/view/download/share + ip_hash
`devices`	Pi. token_hash, agent_version, last_seen_at, last_payload, revoked_at
`heartbeats`	Amostra 15 min / anomalia
`device_events`	press/clip_ok/clip_fail/ffmpeg_restart/boot/queue_drop
`alerts`	Aberto único por (device, kind)
`contracts` / `invoices`	Mensalidade
`equipment` / `equipment_events`	Inventário físico
`venue_access_links`	Link mágico do portal
`audit_log`	Ações do portal/admin

Seed: `venue-demo` "Arena Demo"; court `demo01` "Quadra 01"; `venues.public_key = demo` -> `/g/demo`.

`0002_replay_requests` foi substituído pelo fluxo device/slot; a tabela foi dropada em 0003.

10. Autenticação e segurança (o que o código faz)

- Token do Pi nunca é persistido em claro; só SHA-256.
- Admin token comparado em tempo constante (hash + `timingSafeEqual`).

- Sessão admin: HMAC do expiry com chave derivada do ADMIN_TOKEN - trocar o token invalida cookies.
- Link da arena: token aleatório, guarda hash; Referrer-Policy no redirect; cookie de portal.
- API local do agent bound a 127.0.0.1 - não entra no túnel Cloudflare.
- R2 privado; Range requests no GET do MP4.
- Logo sem SVG.
- IP de analytics hasheado com sal + dia.
- `/painel` e `/arena` mandam `noindex, nofollow`.
- Galeria `/q` e `/g` são públicas: é o produto do QR.

11. Observabilidade

- Worker: observability + traces no wrangler.
- Painel: device online se last_seen < 180 s; issues do último payload.
- Cron 5 min: alerta persistido (mesmo sem e-mail).
- Pi: journalctl; health local; temp/throttle no heartbeat.

Com e-mail desligado, "envio" retorna sucesso no log `email_skipped` para o alerta não ficar pendente para sempre.

12. Mapa do repositório (implementado)

Path	Conteúdo
`src/index.ts`	Worker: público + edge + crons + OG
`src/admin.ts`	API do painel
`src/arena.ts`	API do portal
`src/auth.ts`	edge / admin / arena sessions
`src/jobs.ts`	alertas, purge, faturas, resumo
`src/queries.ts`	SQL compartilhado, brand, issues
`src/lib.ts`	BRT, hash, retention
`web/src/pages/*`	Home, Court, Venue, Clip, Qr
`web/src/admin/*`	Painel
`web/src/arena/*`	Portal
`edge/agent.mjs`	Borda
`edge/pi/*`	install, unit, udev, update.ps1
`firmware/button/button.ino`	ESP32
`migrations/0001-0008`	Schema
`scripts/create-device.mjs`	Emite EDGE_TOKEN
`wrangler.jsonc`	Bindings e crons

Também existem `edge/loadtest-6cam.mjs` e scripts e2e no repo (ferramentas de teste, não o serviço de produção).

13. Operação do piloto - checklist

1. Pi ligada, Wi-Fi ok, cabo no switch, cam no PoE.
2. `systemctl is-active replay-agent` = active.
3. `curl 127.0.0.1:8788/health` -> `buffer_alive: true`, `busy: false`. Após boot, ~25 s de warm.
4. ESP em `/dev/replay-button`. LED GPIO 2 fixo = HB/ACK.
5. Aperto: GPIO 4 no GND. LED pisca -> clip -> 3 piscadas.
6. Conferência: `/q/demo01` ou `/g/demo`.
7. Simular sem botão: `POST http://127.0.0.1:8788/local/replay` via SSH.

Offline no painel: 3 min sem heartbeat. Alerta de cron: 10 min. Filas: 0 no health = upload em dia.

Desligar Pi/câmeras é seguro. Clips já no R2 continuam até expirar. Ao religar, o agent aquece o buffer e o alerta offline fecha sozinho no cron.

> Limite explícito deste documento. Não descreve live (YouTube/Instagram), NVR 24h, transcode na nuvem, troca da Pi, VLAN, nem segundo agent em systemd. Isso não está implementado. O que está no ar é o pipeline botão -> buffer -> MP4 -> R2 -> galeria 7 dias, com painel e portal da arena.

Fim do guia. Código de referência: `edge/agent.mjs` v0.4.5, `firmware/button/button.ino`, `src/index.ts`, `wrangler.jsonc`, migrações 0001-0008.